

PYTHON ASSIGNMENT 2 – PROJECT REPORT

Data Structures – List, Dictionary, Set & Conditional Statements

1. Project Title

Python Data Structures and Conditional Statements

2. Introduction

This project focuses on understanding and implementing fundamental Python data structures and conditional statements. The assignment covers four important concepts: **Lists, Dictionaries, Sets, and Conditional Statements.**

3. List Operations

3.1 List Creation

Two lists were created as part of the assignment:

- age_list – containing five integer values.
- name_list – containing five string values.

Lists are ordered and mutable collections in Python.

The assignment requires creating both an age list and a name list.

3.2 List Modification

Several list methods were used to modify the lists.

Append

The append() method was used to add "Yazhini" to the name_list.

```
name_list.append("Yazhini")
```

Insert

The insert() method was used to add 30 at index 2 in the age_list.

```
age_list.insert(2, 30)
```

Remove

The remove() method was used to remove "Yazhini" from the name_list.

```
name_list.remove("Yazhini")
```

Pop

The pop() method was used to remove the last element from the age_list.

```
age_list.pop()
```

Extend

The extend() method was used to add additional ages.

```
age_list.extend([29, 30, 26])
```

Sort

The sort() method with reverse=True was used to arrange the ages in descending order.

```
age_list.sort(reverse=True)
```

These operations are specifically required in the assignment.

3.3 Finding Maximum, Minimum and Sum

Python's built-in functions were used to analyse the values in age_list.

```
max_age = max(age_list)
```

```
min_age = min(age_list)
```

```
total_age = sum(age_list)
```

The functions perform the following operations:

- max() – finds the largest value.
 - min() – finds the smallest value.
 - sum() – calculates the total of all values.
-

4. Accessing List Elements

Different techniques were used to access elements from name_list.

4.1 Accessing the First Element

```
print(name_list[0])
```

Index 0 represents the first element in a Python list.

4.2 Accessing the Last Element

```
print(name_list[-1])
```

The index -1 represents the last element.

4.3 Accessing a Range of Elements

List slicing was used to access elements from index 2 to index 4.

```
print(name_list[2:5])
```

Python excludes the ending index in slicing, so index 5 is used to include the element at index 4.

4.4 Reversing the List

The list was displayed in reverse order using slicing.

```
print(name_list[::-1])
```

These list-accessing operations satisfy the requirements specified in the assignment.

5. Dictionary Operations

5.1 Dictionary Creation

A dictionary named student_marks was created containing the names and marks of five students.

5.2 Accessing a Dictionary Value

The mark of a particular student was accessed using the student's name.

```
print(student_marks["Arun"])
```

This returns the mark associated with the key "Arun".

5.3 Adding a New Student

A new student named "Janani" was added with a mark of 80.

```
student_marks["Janani"] = 80
```

This demonstrates how new key-value pairs can be added to a dictionary.

5.4 Updating a Student's Mark

An existing student's mark was updated to 82.

```
student_marks["Arun"] = 82
```

This demonstrates that dictionary values can be modified using their keys.

5.5 Dictionary Keys

The keys() method was used to display all student names.

```
print(student_marks.keys())
```

5.6 Dictionary Values

The values() method was used to display all marks.

```
print(student_marks.values())
```

5.7 Dictionary Items

The items() method was used to display all key-value pairs.

```
print(student_marks.items())
```

The assignment specifically requires the use of `keys()`, `values()`, and `items()`.

6. Set Operations

6.1 Creating a Set

A set named `my_set` was created using the given vowel values.

```
my_set = {'a', 'e', 'i', 'o', 'u', 'a', 'a', 'i'}
```

```
print(my_set)
```

6.2 Attempting to Modify a Set Element

The following operation was attempted:

```
my_set[4] = 's'
```

This produces a `TypeError`.

Explanation

A set does not support indexing or item assignment. Therefore, individual elements cannot be changed using an index.

The assignment specifically requires attempting this operation and explaining the resulting error.

6.3 Creating Two Sets

Two sets were created:

```
set1 = {1, 3, 5, 7, 9}
```

```
set2 = {2, 3, 5, 8, 10}
```

6.4 Union Operation

The union operation combines all unique elements from both sets.

```
union_set = set1.union(set2)
```

```
print(union_set)
```

The union contains:

```
{1, 2, 3, 5, 7, 8, 9, 10}
```

6.5 Intersection Operation

The intersection operation identifies the values common to both sets.

```
intersection_set = set1.intersection(set2)
```

```
print(intersection_set)
```

The common values are:

```
{3, 5}
```

The assignment requires both union and intersection operations.

7. Conditional Statements

The final section of the assignment focuses on conditional statements using:

- if
- elif
- else

The purpose is to classify a performance score entered by the user.

8. Performance Category Program

8.1 Input

The user is asked to enter a score between **0 and 10**, inclusive.

```
score = float(input("Enter your score (0 to 10): "))
```

8.2 Performance Criteria

The assignment defines three performance categories:

Score	Performance Category
Greater than 7	Above Average
4 to 7	Average
Less than 4	Below Average

These criteria are specified in the assignment.

8.3 Conditional Program

```
score = float(input("Enter your score (0 to 10): "))
```

```
if score > 7:
```

```
    print("Above Average: Excellent performance! Keep up the good work.")
```

```
elif score >= 4:
```

```
    print("Average: Good effort! Keep practicing, there's room for improvement.")
```

```
else:
```

```
    print("Below Average: Need to improve your performance. Consistent practice will lead to better results.")
```

8.4 Program Output

Example 1 – Above Average

Input:

8

Output:

Above Average: Excellent performance! Keep up the good work.

Example 2 – Average

Input:

7

Output:

Average: Good effort! Keep practicing, there's room for improvement.

The assignment also provides a score of 7 as an example of the Average category.

Example 3 – Below Average

Input:

3

Output:

Below Average: Need to improve your performance. Consistent practice will lead to better results.

9. Key Python Concepts Learned

Through this assignment, the following concepts were practiced:

Lists

- List creation
- `append()`
- `insert()`
- `remove()`
- `pop()`
- `extend()`
- `sort()`
- `max()`
- `min()`
- `sum()`

- Indexing
- Slicing
- Reversing

Dictionaries

- Dictionary creation
- Key-value pairs
- Accessing values
- Adding elements
- Updating values
- keys()
- values()
- items()

Sets

- Set creation
- Removing duplicate values
- Set immutability through indexing
- Union
- Intersection

Conditional Statements

- if
 - elif
 - else
 - User input
 - Comparison operators
 - Decision making
-

10. Conclusion

This project provided practical experience with some of the most important fundamental concepts in Python programming. Lists were used for storing and manipulating ordered collections of data, while dictionaries were used to store information in key-value pairs.

Set operations demonstrated how Python handles unique values and how mathematical operations such as union and intersection can be performed. Finally, conditional statements were used to develop a simple decision-making program that categorizes performance based on a user's score.

Overall, the assignment helped strengthen the understanding of Python data structures, built-in methods, indexing, slicing, set operations, dictionaries, and conditional logic. These concepts form an important foundation for further learning in Python programming and data analysis.